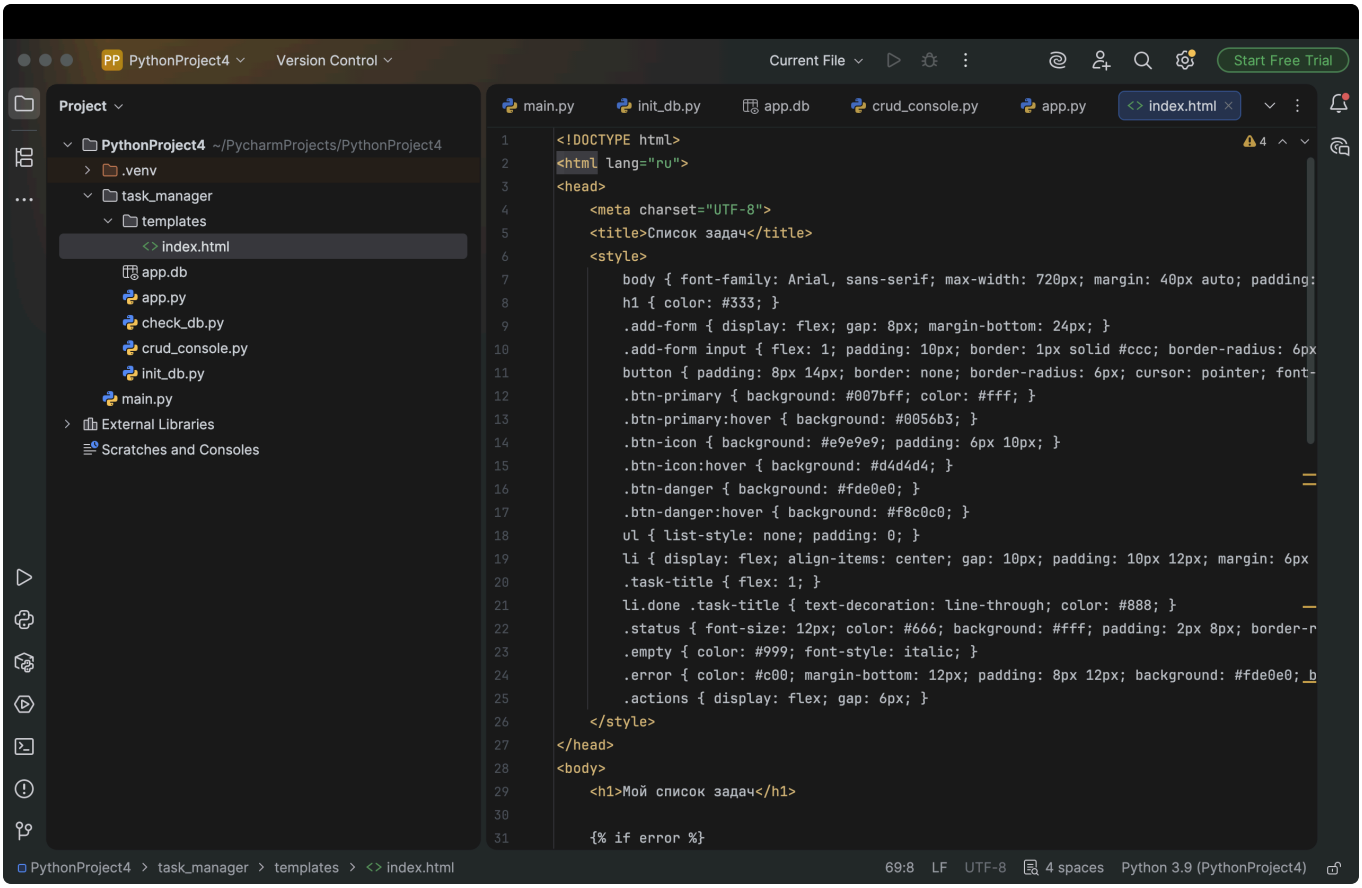
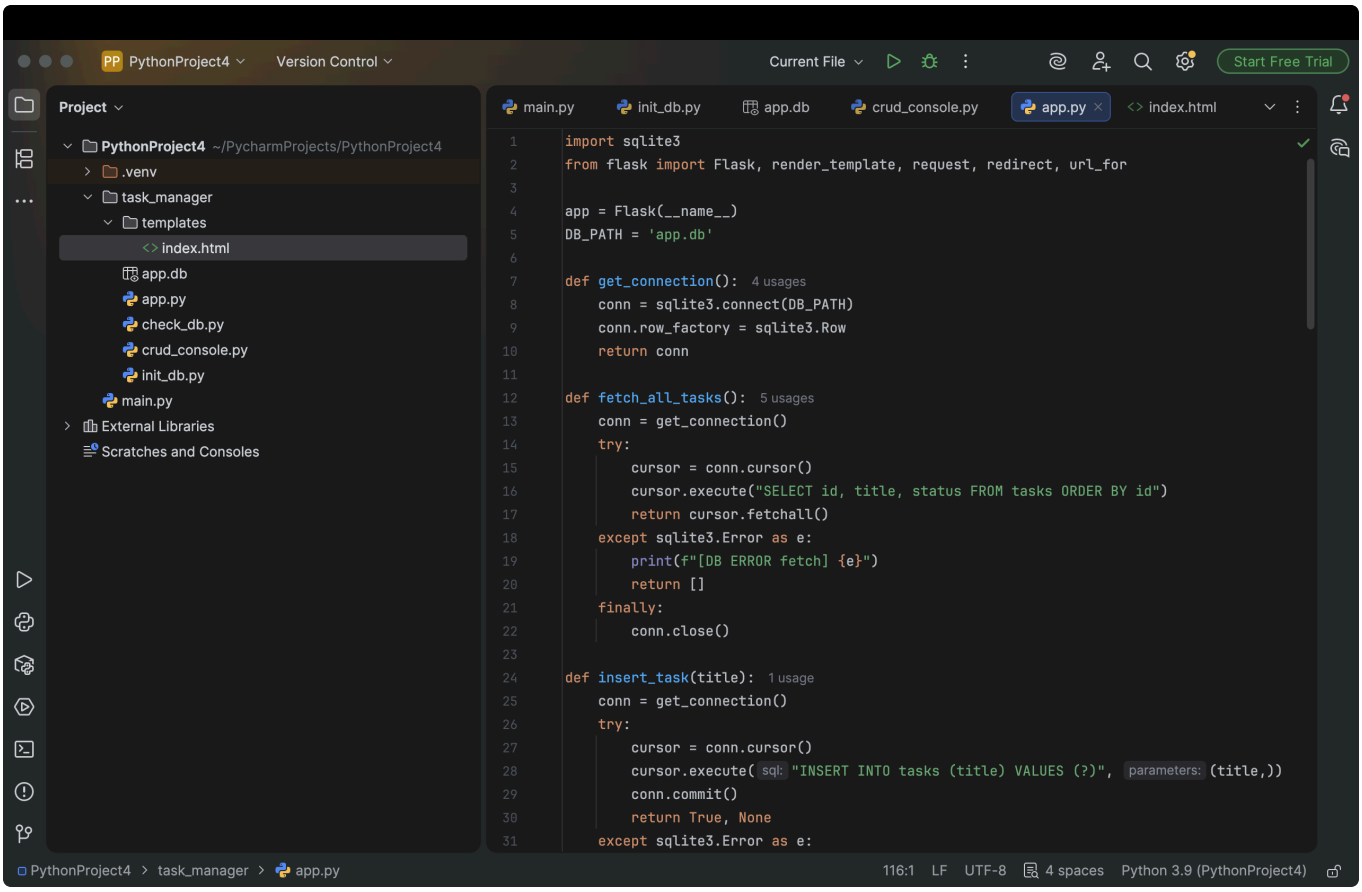
Мини-проект «Список задач» (итоговая работа)

Ястребкова Лилия ЗИСП11-44

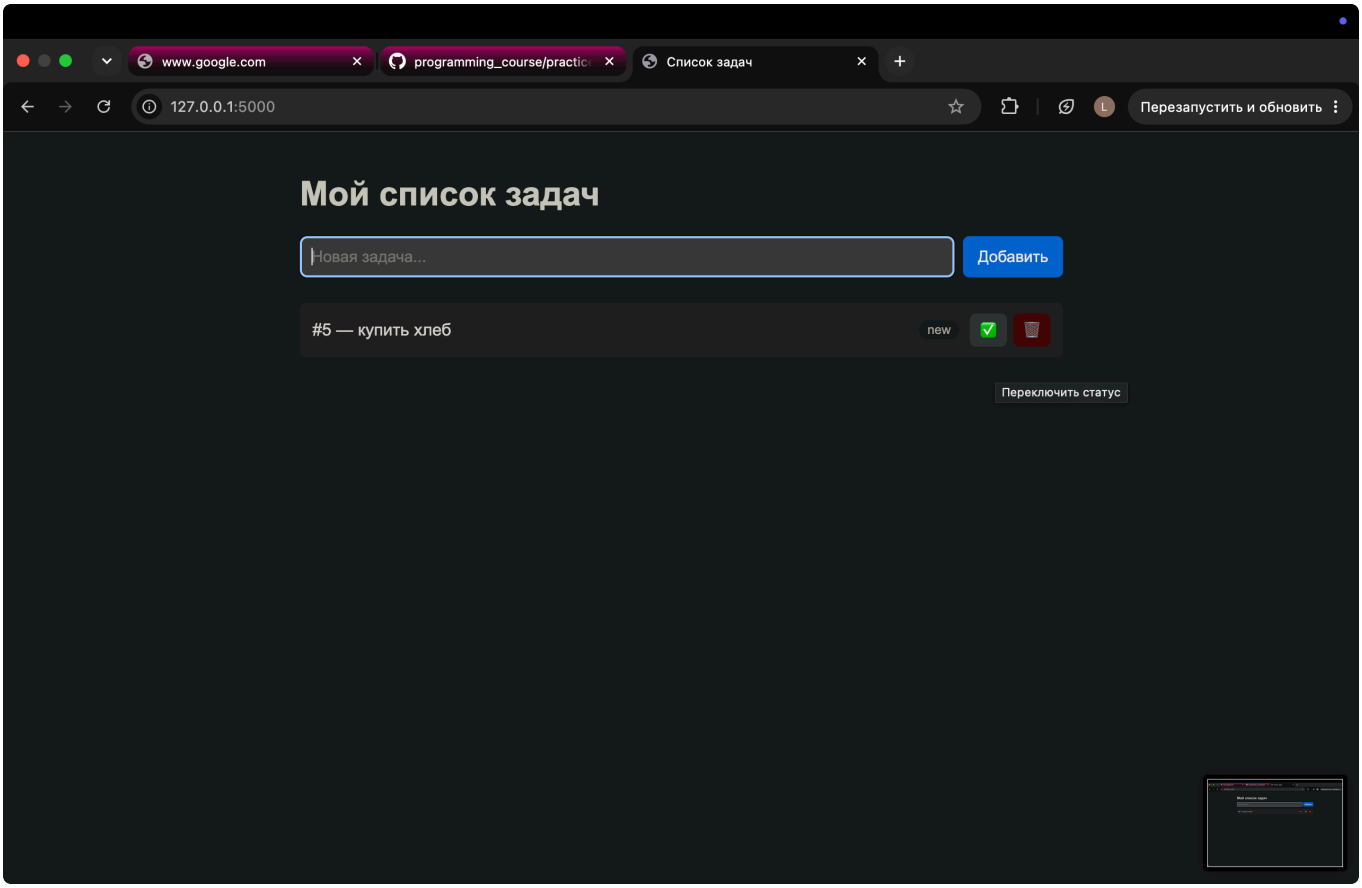
Шаг 2. Новый шаблон `templates/index.html`

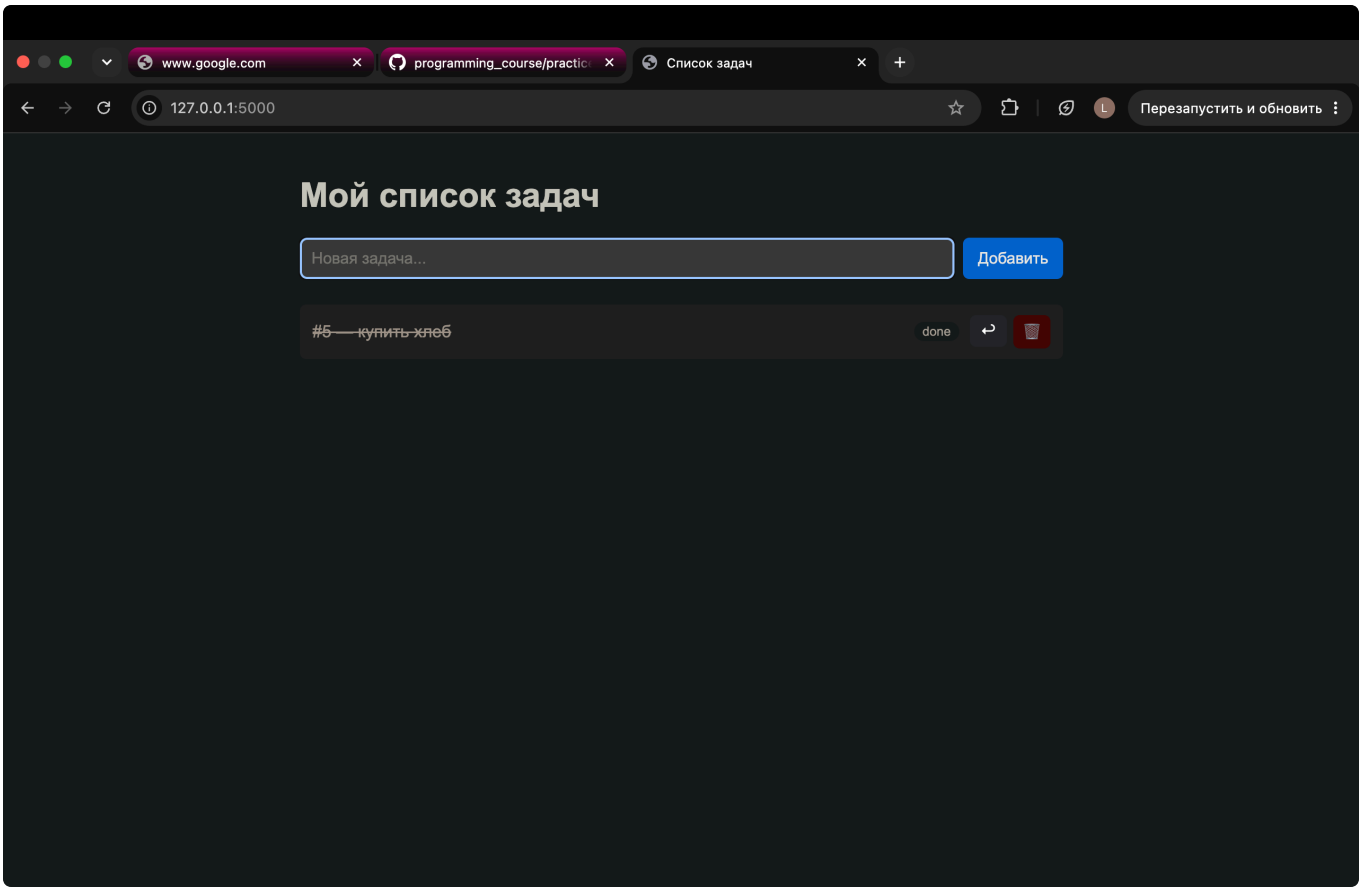


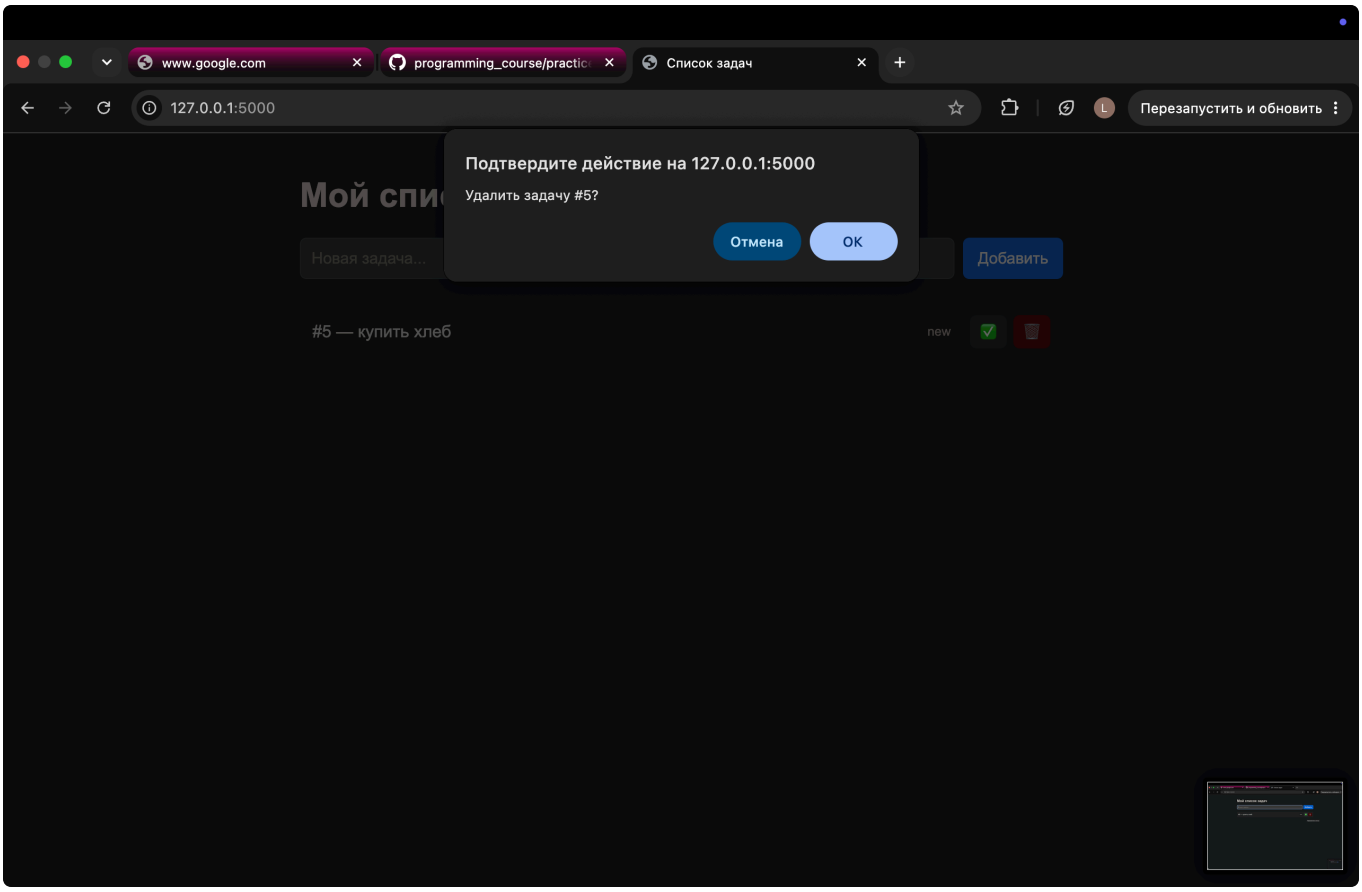
Шаг 3. Полный новый `app.py`

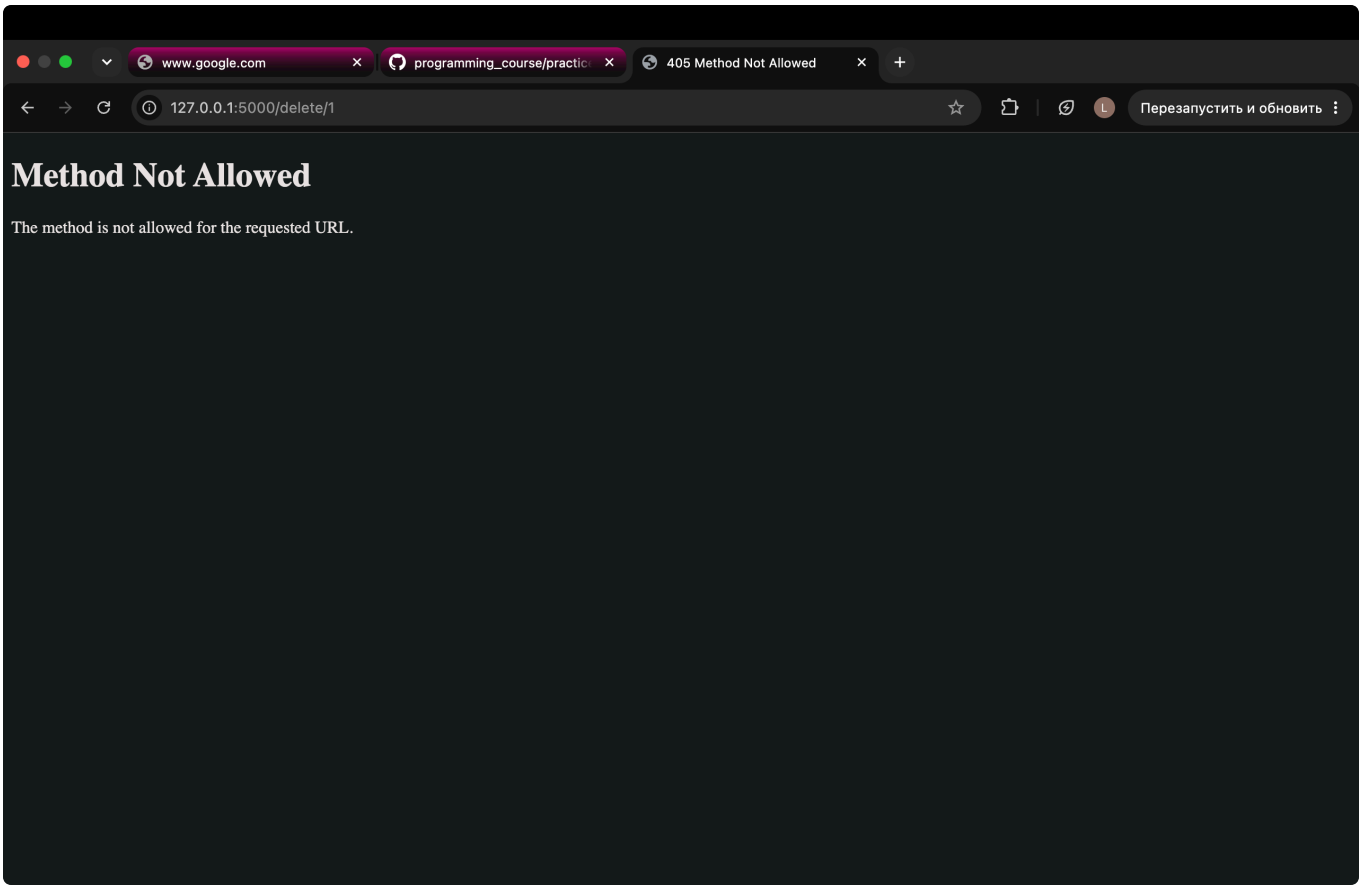


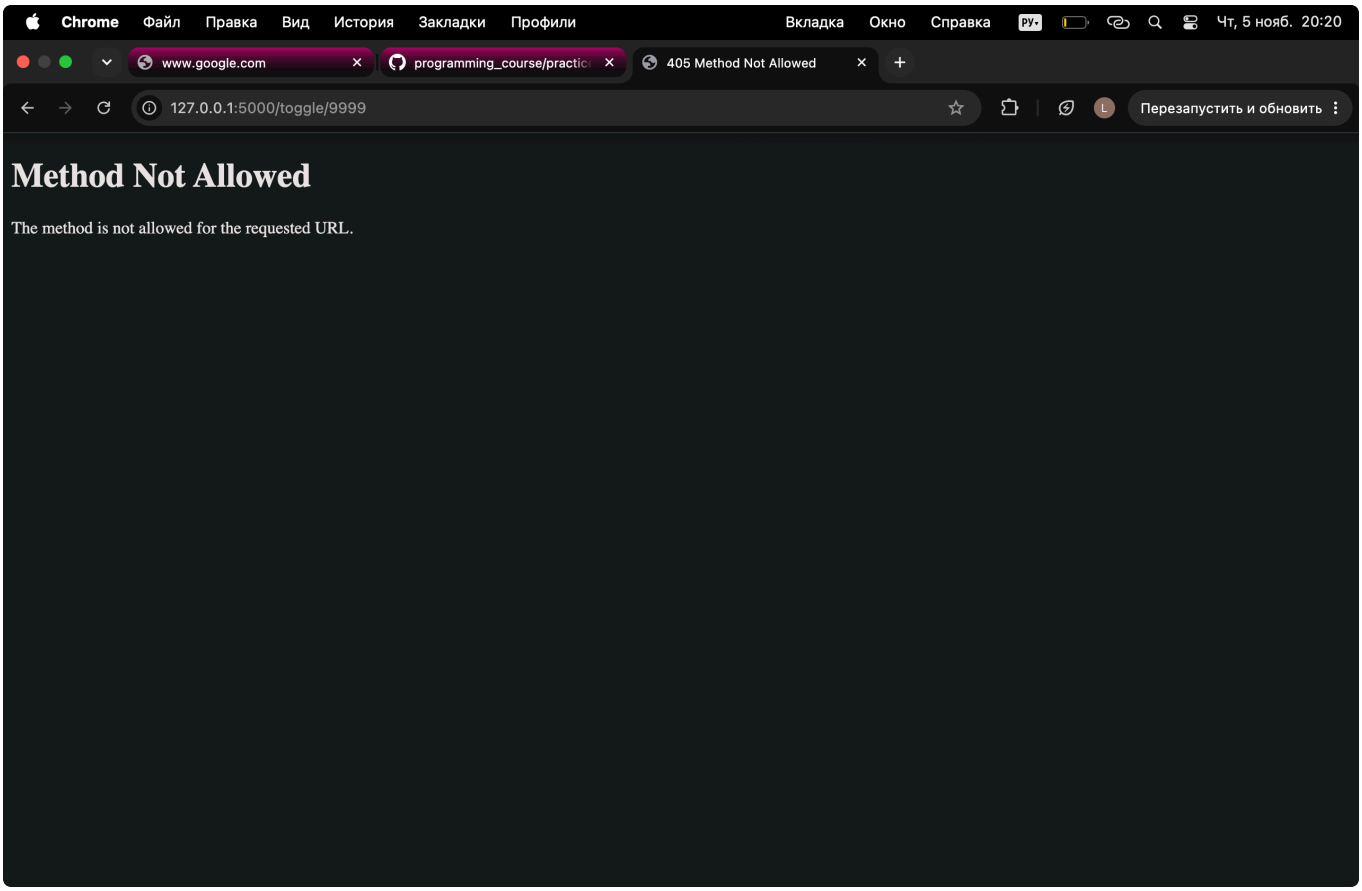
Шаг 4. Запуск и базовый сценарий




www.google.com
programming_course/practic
Список задач
127.0.0.1:5000
Перезапустить и обновить
Мой список задач
Новая задача...
Добавить
#5 — купить хлеб
done


www.google.com
programming_course/practic
Список задач
127.0.0.1:5000
Перезапустить и обновить
Подтвердите действие на 127.0.0.1:5000
Удалить задачу #5?
Отмена
OK
Мой спи
Новая задача...
Добавить
#5 — купить хлеб
new


www.google.com
programming_course/practic
405 Method Not Allowed
127.0.0.1:5000/delete/1
Перезапустить и обновить
Method Not Allowed
The method is not allowed for the requested URL.



1. Почему действия «переключить статус» и «удалить» отправляются через POST-форму, а не через `<a href>`?

Ссылка `<a href>` - это GET-запрос, а по семантике HTTP GET не должен менять данные на сервере. Обе операции меняют состояние: UPDATE переключает статус, DELETE удаляет запись. Если оформить их ссылками, возникает несколько проблем: браузер может закэшировать GET-ответ, поисковые системы и превью-боты случайно «нажмут» ссылку и удалят записи, а повторное открытие страницы повторит деструктивное действие. POST-форма явно сигнализирует: «это операция, изменяющая данные», и не кэшируется. Кроме того, POST-маршруты принимают только POST - обращение к `/delete/5` через GET вернёт 405, что защищает от случайного выполнения действия.

2. Что делает конвертер `<int:task_id>` в маршруте и почему с ним не нужен `isdigit()`?

`<int:task_id>` - это переменная часть URL: Flask извлекает из адреса фрагмент, приводит его к типу `int` и передаёт в функцию-обработчик как аргумент `task_id`. Если в URL окажется не число (например, `/toggle/abc`), Flask сам вернёт **404 Not Found** ещё до вызова функции - приведение типа не удастся. Раньше в консольном варианте приходилось вручную читать строку из `input()` и проверять `isdigit()` перед `int()`,

потому что пользователь мог ввести что угодно. В веб-версии Flask делает эту проверку на уровне маршрутизации, поэтому ручная валидация не нужна.

3. Что означает `rowcount = 0` после `UPDATE` или `DELETE`?

Это значит, что запрос выполнился без ошибок, но ни одна строка не была затронута — то есть записи с таким id не существует. `rowcount` показывает, сколько строк реально изменила последняя команда. Если бы id был найден, вернулось бы `1` (или больше, если WHERE охватывает несколько строк). Поэтому `rowcount = 0` - сигнал вернуть пользователю сообщение «Задача `#N` не найдена» вместо редиректа с молчаливым «успехом».

4. Почему каждая функция работы с базой открывает своё соединение, а не получает его параметром?

Это делает каждую функцию **самодостаточной**: она сама открывает соединение, сама отвечает за try/except/finally, rollback и close. Такой подход проще для понимания и отладки: у каждой операции понятный жизненный цикл, а утечка соединения локализована внутри одной функции и всегда закрывается в `finally`. Если бы соединение передавалось параметром, вызывающий код должен был бы помнить о его закрытии, и при любом забытом `close()` соединение оставалось бы открытым до завершения запроса. Для учебного проекта с SQLite один файл - соединение на операцию вполне оправдано: SQLite быстро открывает локальный файл, а короткоживущие соединения снижают риск блокировок. В крупных приложениях для переиспользования соединений используют пул соединений, но там иная архитектура.

5. Где в `toggle_task_status` находится `commit()`, и почему без него статус не сохранится после перезапуска?

`commit()` находится внутри блока `try`, сразу после `execute` с `UPDATE` — до `close()` в `finally`. Без него изменения остаются в памяти соединения: транзакция «висит» незавершённой, и при закрытии соединения (а `finally` закрывает его всегда) SQLite откатывает незакоммиченные данные. Список задач в браузере при следующем GET-запросе показал бы старый статус, а после перезапуска сервера изменение вообще исчезло бы. `commit()` - единственный способ гарантированно записать изменения в файл `app.db` на диске.

6. Что произойдёт, если открыть `/delete/3` в адресной строке браузера (GET)?

Браузер отправит GET-запрос, а маршрут объявлен как `methods=['POST']` — он GET не принимает. Flask ответит **405 Method Not Allowed**, и задача удалена не будет. Это правильное поведение: деструктивные действия защищены от случайного запуска (например, если кто-то поделится ссылкой или произнесёт URL вслух). Реально удалить

задачу можно только нажатием кнопки, которая отправляет POST-форму с подтверждением через `confirm()`.

7. Зачем в шаблоне `onsubmit="return confirm(...)"` и можно ли обойти эту защиту?

`onsubmit="return confirm(...)"` - браузерное подтверждение перед отправкой формы: если пользователь нажмёт «Отмена», submit отменяется (функция вернёт `false`), и DELETE не выполнится. Это защита от случайного клика по кнопке - необратимого действия. Обойти её можно: подтверждение существует только в JavaScript на клиенте, а напрямую отправить POST на `/delete/3` можно через curl, Postman или самописный скрипт - запрос выполнится без вопроса. Это нормально для учебного проекта: подтверждение - про UX, а не про безопасность. Реальная защита от нежелательных действий реализуется на сервере (проверка прав, токены CSRF, ограничение доступа), а не в браузерном диалоге.

8. Почему `return redirect(url_for('index'))` стоит после `try/finally`, а не внутри `try`?

Технически редирект можно поставить и внутри `try` после `commit` - результат будет одинаковым, потому что `finally` гарантированно выполняется перед фактическим выходом из функции, а `except` перехватит ошибку и вернёт форму с сообщением. Но размещение после блока делает логику читабельнее: `try` целиком отвечает за транзакцию с базой, а возврат ответа - за HTTP-слой. Так понятнее, что `redirect` выполняется **только при успехе**: если `execute` или `commit` упали, управление уходит в `except`, который возвращает страницу с ошибкой и никогда не доходит до редиректа. Если бы редирект стоял внутри `try` до перехвата, при ошибке он просто не выполнился бы - но код был бы менее наглядным.